Milestone 6, Part 3: Training a No-Limit Blueprint

holdem-cfr walkthrough

The abstract game

A blueprint is an approximate equilibrium of a smaller game that stands in for the real one. Our abstract game keeps the real rules, the real blinds and 100-big-blind stacks, and changes only what a player can *say* and what a player can *see*.

What a player can say. At every decision the menu is: fold, check or call, a few pot-fraction raises for that street, and all-in. The default menu, `ActionAbstraction::default_100bb()`, is:

Street	Bets (nothing to call)	Raises (facing a bet, on the pot after calling)	Max bets + raises
Preflop	1.0 pot	0.75, 1.25 pot	4
Flop	33%, 75%	100%	3
Turn	66%	100%	2
River	66%, 125%	100%	2

All-in is always on the menu until the cap is reached. In chips, the button's first decision is fold, limp, raise to 2.5bb, raise to 3.5bb, or shove. Facing an open to 2.5bb, the big blind can 3-bet to 6.25bb or 8.75bb.

The important design choice is that every abstract action is generated as a pot fraction and then applied through the real rules engine from part 1. `State::apply` returns an error for anything illegal, so building the tree *is* a legality check. A raise that would fall below the minimum is clamped up to it. Raises that land on the same amount are merged. A size that exceeds the stack becomes the all-in.

The tree has 64 preflop decision nodes, 466 on the flop, 1,284 on the turn and 3,712 on the river: 13,704 nodes in total, counting terminals. A test pins these counts, so any change to the menu shows up as a test diff.

What a player can see. On each street the player observes their bucket from part 2: 169 exact preflop classes, then 50 E[HS²] or HS percentiles. Regrets and strategies live in flat arrays indexed by (node, bucket, action). With 169/50/50/50 buckets that comes to 707,641 entries, about 6 MB for regrets and strategy sums together. That is small enough to train well in minutes and leaves plenty of room to grow the menu or the bucket counts later.

One trainer, already proven

Training uses the external-sampling MCCFR from part 2, which was validated on Leduc against exact best responses. The no-limit game plugs in through the same `SampledGame` trait. It provides:

- The betting tree. - A sampler that deals nine cards and precomputes both players' buckets on all four streets and both players' seven-card ranks. That is everything the traversal needs, computed once per deal. - A utility function that reads the stored contributions at a terminal node. A fold pays the folder's contribution to the other player. A showdown pays the shared stake to the better rank, or nothing on a tie.

Because the traversal code is literally the code that converged on Leduc, the remaining risk is in the glue, and that is what the tests here target.

Linear CFR without overflowing precision

Pluribus weighted later iterations more heavily (Linear CFR) at the start of training. Doing this by multiplying each update by its iteration number t makes the sums grow like t^2, and once sums reach around 10^{16}, adding a 10^8-sized update to a 32-bit float changes nothing. The fix is the one Pluribus used. Train in batches with uniform weights, and after batch T multiply every regret and strategy sum by T/(T + 1). That gives the same relative weighting while keeping magnitudes bounded. The trainer applies this discount for the first few minutes (`--linear-minutes`) and then switches to plain averaging.

A faster indexer along the way

Profiling the sampler pointed at the isomorphism indexer's hash-map lookup of the suit configuration. Each suit's shape (cards per round) packs into a small integer, and the four sorted shapes into one key, so the map became a dense lookup table. For the board-as-a-set indexers the table has 104,976 entries. The flop bijection test, which indexes all 26 million flop situations, went from 23 seconds to 11.5, and it still passes. That is the point of having it.

The gate

`hunl/tests/blueprint.rs` checks four things:

- **Tree consistency.** The node counts above. Every decision has at least two actions. Children are stored after parents. Contributions stay within the stack. Every showdown has equal contributions. Every terminal pays no more than was put in. - **Valid deals.** Ten thousand sampled hands each have nine distinct cards and in-range buckets. - **Learning.** After 1.5 million iterations using the cheap made-hand-category abstraction, the button plays AA, KK, AKs, AKo and QJs more than 95% of the time

and plays the five worst offsuit hands less than half the time. A heads-up button that folds aces, or plays 72o as often as aces, means a broken utility or bucket. This test runs in CI in about 11 seconds. - **Checkpoints.** Saving and reloading reproduces every regret and strategy sum exactly.

What training looks like

After 5 million iterations, even the crude category abstraction produces the familiar shape of a heads-up button strategy: almost every hand is played, and folds are concentrated in weak offsuit hands like 72o, 82o, 62o and 32o. With the real E[HS²] buckets, the 2-core VM runs about 95,000 iterations per second. An 8-core laptop should run several hundred thousand, so a 20-minute run is a few hundred million iterations.

```
cargo run --release -p cli -- blueprint --minutes 20 --linear-minutes 5
```

The command prints progress after every batch. It writes a resumable checkpoint to `data/blueprint.bin` and, at the end, prints the button's play and raise charts as 13×13 grids: pairs on the diagonal, suited hands above, offsuit below.

What this does and does not establish

The tests establish that the abstract game is legal and consistent, and that the trainer, which is proven on Leduc, learns sensible behaviour here. They do **not** establish how good the blueprint is in the real game. That cannot be computed exactly: a best response to a no-limit strategy is far beyond a laptop, and even within the abstraction, imperfect-recall buckets make exact best responses expensive. Part 4 gives the honest measurements available instead:

- local best response (Lisý and Bowling 2017), a lower bound on exploitability; - head-to-head matches against simple baseline bots, with AIVAT variance reduction; - matches against the bot's own blueprint once real-time re-solving is added.